

Data Warehousing and Data Mining

Unit - 1

Topics Covered

- Overview
- Definition
- Delivery Process
- Difference between Database System and Data Warehouse
- Multi-Dimensional Data Model
- Data Cubes
- Stars
- Snow Flakes
- Fact Constellations
- Concept hierarchy
- Process Architecture
- 3 Tier Architecture
- Data Marting

Prerequisite

What is Database?

A database is a **structured collection of data** that is **stored and managed** in a way that makes it **easy to retrieve, update, and manipulate**.

Databases are used in **various applications**, ranging from **simple contact lists** to complex systems like **online banking and social media platforms**.

Key points about Database:

1. **Structure:** Databases are typically organized into tables, which consist of **rows and columns**.
2. **Database Management Systems (DBMS):** This is the **software** used to manage and interact with the database.

Popular DBMSs include **MySQL, PostgreSQL, Oracle, and Microsoft SQL Server**.

3. **ER-Model** : Entity-Relationship (ER) model is a way to visually represent how different pieces of data relate to each other in a database. It's like a blueprint for designing a database.
4. **Normalization**: This is the database design process goal is to organize the data efficiently, which often involves dividing a database into multiple related tables.

WHAT IS DATA WAREHOUSE ?

- A data warehouse is a centralized repository that stores large(TB) volumes of historical data from various sources.
- It is optimized for complex queries and data analysis, supporting business intelligence and decision-making.
- Unlike databases, which handle current transactions, data warehouses focus on long-term data aggregation and trend analysis.

DIFFERENCE BETWEEN DATABASE AND DATAWAREHOUSE

Database	Datawarehouse
Primarily designed for real-time transactional processing .	Designed for analytical processing and reporting .
Handles day to day current Transactions such as inserting, updating, and deleting data.	Aggregates data from various sources to support business intelligence and decision-making.
Optimized for speed and efficiency in querying and accessing .	Optimized for complex querying and analyzing .
Uses a normalized structure (Relational model) to minimize redundancy and ensure data integrity.	Often uses a denormalized structure (e.g., star schema or snowflake schema) to optimize query performance.

DIFFERENCE BETWEEN DATABASE AND DATAWAREHOUSE

Database	Datawarehouse
Data is organized in tables with relationships defined by primary and foreign keys .	Data is usually organized into Dimension modelling (facts and dimensions) to support multidimensional analysis.
Generally stores data from a single application or source .	Integrates data of same subject (Finance, Slaes, Banking, etc.) from multiple sources , including different databases, external files, and applications .
Contains operational data that is frequently updated .	Contains historical data collected over time , making it suitable for trend analysis and reporting.
Data is updated in real-time as transactions occur.	Data is typically updated in batches (e.g., daily, weekly).

DIFFERENCE BETWEEN DATABASE AND DATAWAREHOUSE

Database	Datawarehouse
Used by applications for daily operations , such as customer relationship management (CRM) systems, enterprise resource planning (ERP) systems, and other transactional systems.	Used by analysts and business intelligence tools for reporting, data mining , and making strategic decisions.

DEFINE DATAWAREHOUSE

According to **Bill Inmon**



A Data warehouse is a **subject oriented, integrated, time-varient & non-volatile** collection of data in support of managements decision making process.

Bill Inmon designs **Top-Down approach**.

DEFINE DATAWAREHOUSE

According to **Ralph Kimball**



A warehouse is a **copy of transaction data** specifically structured for **query & analysis**.

Ralph Kimball designs **Bottom-Up approach**

What is Data Warehouse?

- Defined in many different ways, but not rigorously.
 - A decision support database that is maintained **separately** from the organization's operational database
 - Support **information processing** by providing a solid platform of consolidated, historical data for analysis.
- “A data warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision-making process.” —W. H. Inmon
- Data warehousing:
 - The process of constructing and using data warehouses

Data Warehouse—Subject-Oriented

- Organized around major subjects, such as **customer, product, sales**.
- Focusing on the modeling and analysis of data for decision makers, not on daily operations or transaction processing.
- Provide **a simple and concise** view around particular subject issues by **excluding data that are not useful in the decision support process**.

Data Warehouse—Integrated

- Constructed by integrating multiple, heterogeneous data sources
 - relational databases, flat files, on-line transaction records
- Data cleaning and data integration techniques are applied.
 - Ensure consistency in naming conventions, encoding structures, attribute measures, etc. among different data sources
 - E.g., Hotel price: currency, tax, breakfast covered, etc.
 - When data is moved to the warehouse, it is converted.

Data Warehouse—Time Variant

- The time horizon for the data warehouse is significantly longer than that of operational systems.
 - Operational database: current value data.
 - Data warehouse data: provide information from a historical perspective (e.g., past 5-10 years)
- Every key structure in the data warehouse
 - Contains an element of time, explicitly or implicitly
 - But the key of operational data may or may not contain “time element”.

Data Warehouse—Non-Volatile

- A **physically separate store** of data transformed from the operational environment.
- Operational **update of data does not occur** in the data warehouse environment.
 - Does not require transaction processing, recovery, and concurrency control mechanisms
 - Requires only two operations in data accessing:
 - *initial loading of data* and *access of data*.

Data Warehouse vs. Heterogeneous DBMS

- Traditional heterogeneous DB integration:
 - Build **wrappers/mediators** on top of heterogeneous databases
 - **Query driven** approach
 - When a query is posed to a client site, a meta-dictionary is used to translate the query into queries appropriate for individual heterogeneous sites involved, and the results are integrated into a global answer set
 - Complex information filtering, compete for resources
- Data warehouse: **update-driven**, high performance
 - Information from heterogeneous sources is integrated in advance and stored in warehouses for direct query and analysis

Data Warehouse vs. Operational DBMS

- **OLTP (on-line transaction processing)**
 - **Major task of traditional relational DBMS**
 - **Day-to-day operations: purchasing, inventory, banking, manufacturing, payroll, registration, accounting, etc.**
- **OLAP (on-line analytical processing)**
 - **Major task of data warehouse system**
 - **Data analysis and decision making**

Distinct features (OLTP vs. OLAP):

- **User and system orientation: customer vs. market**
- **Data contents: current, detailed vs. historical, consolidated**
- **Database design: ER + application vs. star + subject**
- **View: current, local vs. evolutionary, integrated**
- **Access patterns: update vs. read-only but complex queries**

OLTP vs. OLAP

	OLTP	OLAP
users	clerk, IT professional	knowledge worker
function	day to day operations	decision support
DB design	application-oriented	subject-oriented
data	current, up-to-date detailed, flat relational isolated	historical, summarized, multidimensional integrated, consolidated
usage	repetitive	ad-hoc
access	read/write index/hash on prim. key	lots of scans
unit of work	short, simple transaction	complex query
# records accessed	tens	millions
#users	thousands	hundreds
DB size	100MB-GB	100GB-TB
metric	transaction throughput	query throughput, response

Why Separate Data Warehouse?

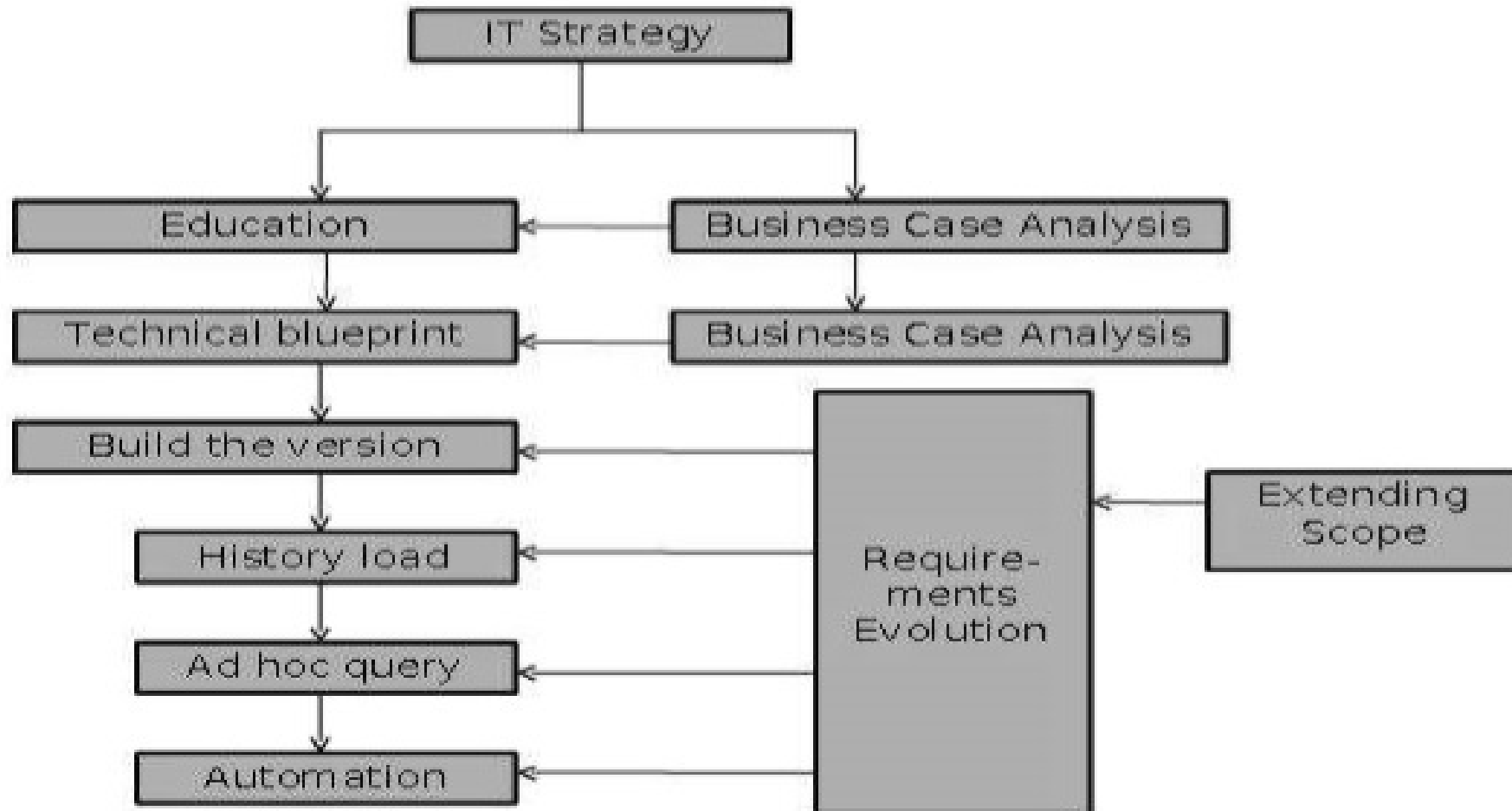
- High performance for both systems
 - DBMS— tuned for OLTP: access methods, indexing, concurrency control, recovery
 - Warehouse—tuned for OLAP: complex OLAP queries, multidimensional view, consolidation.
- Different functions and different data:
 - missing data: Decision support requires historical data which operational DBs do not typically maintain
 - data consolidation: DS requires consolidation (aggregation, summarization) of data from heterogeneous sources
 - data quality: different sources typically use inconsistent data representations, codes and formats which have to be reconciled

Data Warehousing - Delivery Process

- A data warehouse is never static; it evolves as the business expands.
- As the business evolves, its requirements keep changing and therefore a data warehouse must be designed to ride with these changes.
- Hence a data warehouse system needs to be flexible.
- Ideally there should be a delivery process to deliver a data warehouse.
- However data warehouse projects normally suffer from various issues that make it difficult to complete tasks and deliverables in the strict and ordered fashion demanded by the waterfall method.

Delivery Method

- The delivery method is a variant of the joint application development approach adopted for the delivery of a data warehouse. We have staged the data warehouse delivery process to minimize risks



IT Strategy

- Data warehouse are strategic investments that require a business process to generate benefits.
- IT Strategy is required to procure and retain funding for the project.

Business Case

- The objective of business case is to **estimate business benefits that should be derived from using a data warehouse.**
- These benefits **may not be quantifiable** but the projected benefits need to be clearly stated.
- If a data warehouse **does not have a clear business case, then the business tends to suffer from credibility problems** at some stage during the delivery process.
- Need to understand the business case for investment.

Education and Prototyping

- Organizations experiment with the concept of data analysis and educate themselves on the value of having a data warehouse before settling for a solution.
- This is addressed by **prototyping**.
- It helps in understanding the **feasibility and benefits of a data warehouse**.

Education and Prototyping

- The prototyping activity on a **small scale** can promote educational process as long as
 - The prototype addresses a defined technical objective.
 - The prototype can be thrown away after the feasibility concept has been shown.
 - The activity addresses a small subset of eventual data content of the data warehouse.
 - The activity timescale is non-critical.

Education and Prototyping

- The following points are to be kept in **mind to produce an early release and deliver business benefits**.
 - Identify the architecture that is capable of evolving.
 - Focus on business requirements and technical blueprint phases.
 - Limit the scope of the first build phase to the minimum that delivers business benefits.
 - Understand the short-term and medium-term requirements of the data warehouse.

Business Requirements

- To provide quality deliverables, we should make sure the overall requirements are understood.
- If we understand the business requirements for **both short-term and medium-term**, then we can design a solution to fulfil short-term requirements.
- The following aspects are determined in this stage –
 - The business rule to be applied on data.
 - The logical model for information within the data warehouse.
 - The query profiles for the immediate requirement.
 - The source systems that provide this data.

Technical Blueprint

- This phase need to deliver an **overall architecture satisfying the long term requirements.**
- This phase also deliver the components that must be implemented in a **short term to derive any business benefit.**
- The blueprint need to identify the followings.
 - The overall system architecture.
 - The data retention policy.
 - The backup and recovery strategy.
 - The server and data mart architecture.
 - The capacity plan for hardware and infrastructure.
 - The components of database design.

Building the Vision

- In this stage, the first production deliverable is produced.
- This production deliverable is the smallest component of a data warehouse.
- This smallest component adds business benefit.
- This stage will probably build the large components for extracting and loading.

History Load

- Remaining phases- in any order
- This is the phase where the remainder of the required history is loaded into the data warehouse.
- In this phase, we do not add new entities, but additional physical tables would probably be created to store increased data volumes.

Scenario:

Suppose the build version phase has delivered a **retail sales analysis** data warehouse with 2 months' worth of history.

This information will allow the user to **analyze only the recent trends** and address the short-term issues.

The user in this case **cannot identify annual and seasonal trends**.

To help him do so, last 2 years' sales history could be loaded from the archive. Now the 40GB data is extended to 400GB.

Ad hoc Query

- In this phase, we configure an **ad hoc query tool** that is used to operate a data warehouse.
- These tools can generate the database query automatically from the user request

Automation

- In this phase, operational management processes are fully automated. These would include
 - Transforming the data into a form suitable for analysis.
 - Monitoring query profiles and determining appropriate aggregations to maintain system performance.
 - Extracting and loading data from different source systems.
 - Backing up, restoring, and archiving the data.
 - Generating aggregations from predefined definitions within the data warehouse.
 - Monitoring query profiles

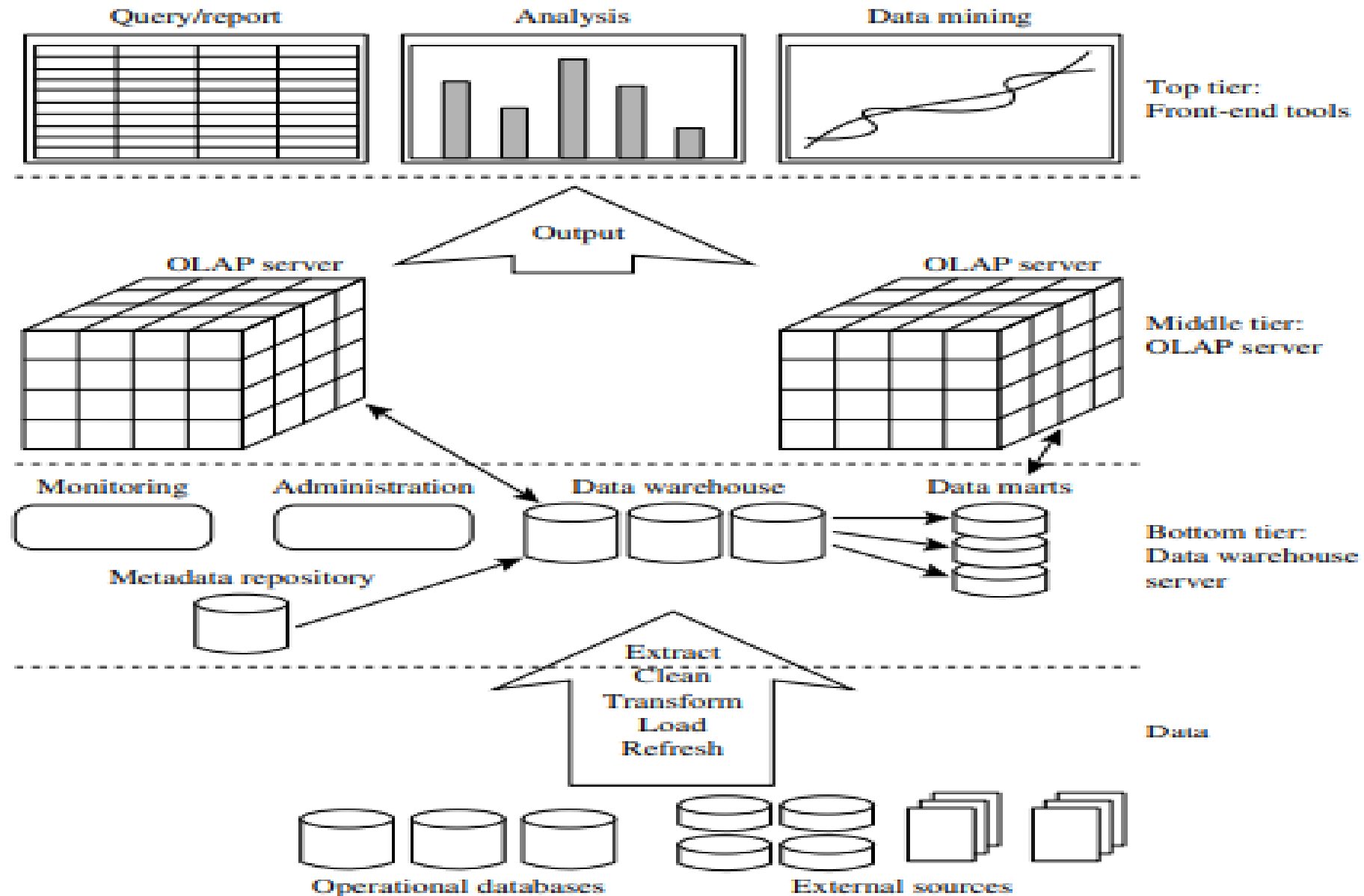
Extending Scope

- In this phase, the data warehouse is extended to address a new set of business requirements.
- The scope can be extended in two ways
 - By loading additional data into the data warehouse.
 - By introducing new data marts using the existing information.

Requirements Evolution

- From the perspective of delivery process, the requirements are always changeable. **They are not static.**
- The delivery process must support this and allow these changes to be reflected within the system.
- The **architecture is designed to change and grow to match the business needs**, the process operates as a pseudo-application development process, where the new requirements are continually fed into the development activities and the partial deliverables are produced.
- These **partial deliverables are fed back** to the users and then reworked ensuring that the overall system is continually updated to meet the business needs.

Data Warehousing: A Multitiered Architecture



Bottom Tier – Data Storage and Integration

- This foundational layer comprises a relational database system that serves as the central repository for data.
- Data is extracted from various operational databases and external sources using back-end tools and utilities.
- These tools perform data extraction, cleaning, transformation, loading, and refreshing to ensure the data warehouse remains up-to-date.
- The bottom tier also includes a metadata repository, which stores information about the data warehouse's contents and structure.

Middle Tier – OLAP Engine

- The middle tier houses the Online Analytical Processing (OLAP) server, which facilitates complex analytical queries and data modeling. There are two primary types of OLAP servers:
- **ROLAP (Relational OLAP):** Utilizes relational databases to store and manage warehouse data, translating multidimensional operations into relational queries.
- **MOLAP (Multidimensional OLAP):** Employs multidimensional databases, allowing for faster retrieval of summarized data.

Front-end client layer

- Contains query and reporting tools, analysis tools, and/or data mining tools (e.g., trend analysis, prediction, and so on).

A multi-dimensional data model

Data Cube

- A data cube allows data to be modeled and viewed in multiple dimensions.
- It is defined by **dimensions and facts**.
- Dimensions are the perspectives or **entities** with respect to which an organization wants to keep records
- **Facts are numeric measures**. Think of them as the quantities by which we want to analyze relationships between dimensions.

Data Cubes

- A data warehouse is based on a **multidimensional data model** which views data in the form of a data cube.
- A data cube, such as **sales**, allows data to be modeled and viewed in multiple dimensions
 - Dimension tables, such as **item (item_name, brand, type)**, or **time(day, week, month, quarter, year)**
 - Fact table contains measures (such as **dollars_sold**) and keys to each of the related dimension tables
- In data warehousing literature, an n-D base cube is called a **base cuboid**. The top most 0-D cuboid, which holds the highest-level of summarization, is called the **apex cuboid**. The lattice of cuboids forms a **data cube**.

From Tables and Spreadsheets to Data Cubes

Table 4.2 2-D View of Sales Data for *AllElectronics* According to *time* and *item*

<i>location</i> = "Vancouver"				
<i>time</i> (quarter)	<i>item</i> (type)			
	<i>home</i> <i>entertainment</i>	<i>computer</i>	<i>phone</i>	<i>security</i>
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Note: The sales are from branches located in the city of Vancouver. The measure displayed is *dollars_sold* (in thousands).

From Tables and Spreadsheets to Data Cubes

Table 4.3 3-D View of Sales Data for *AllElectronics* According to *time*, *item*, and *location*

<i>location</i> = "Chicago"					<i>location</i> = "New York"				<i>location</i> = "Toronto"				<i>location</i> = "Vancouver"			
<i>Item</i>					<i>Item</i>				<i>Item</i>				<i>Item</i>			
<i>home</i>					<i>home</i>				<i>home</i>				<i>home</i>			
<i>time</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Note: The measure displayed is *dollars_sold* (in thousands).

From Tables and Spreadsheets to Data Cubes

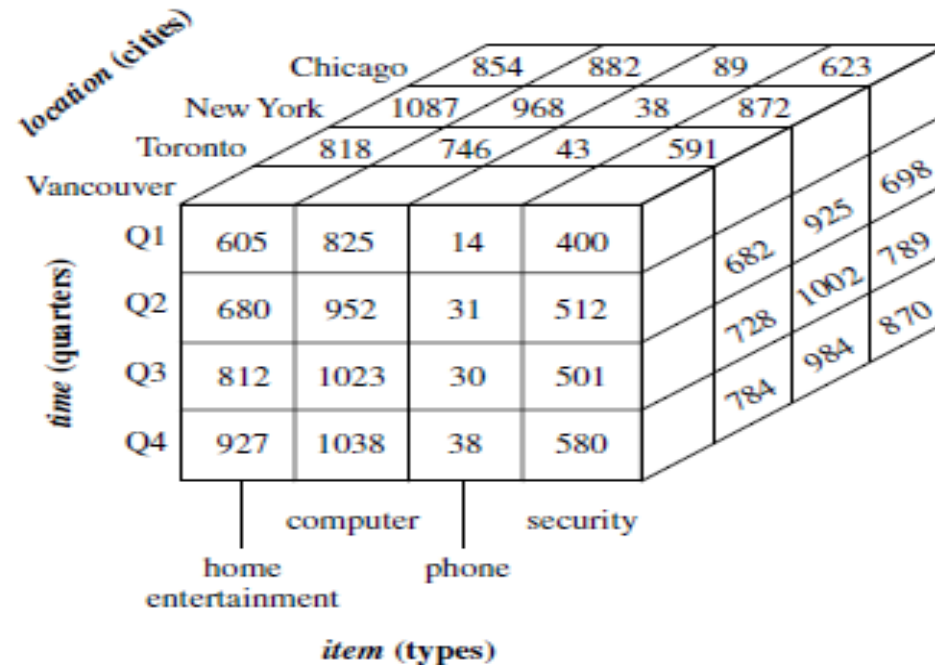


Figure 4.3 A 3-D data cube representation of the data in Table 4.3, according to *time*, *item*, and *location*. The measure displayed is *dollars_sold* (in thousands).

From Tables and Spreadsheets to Data Cubes

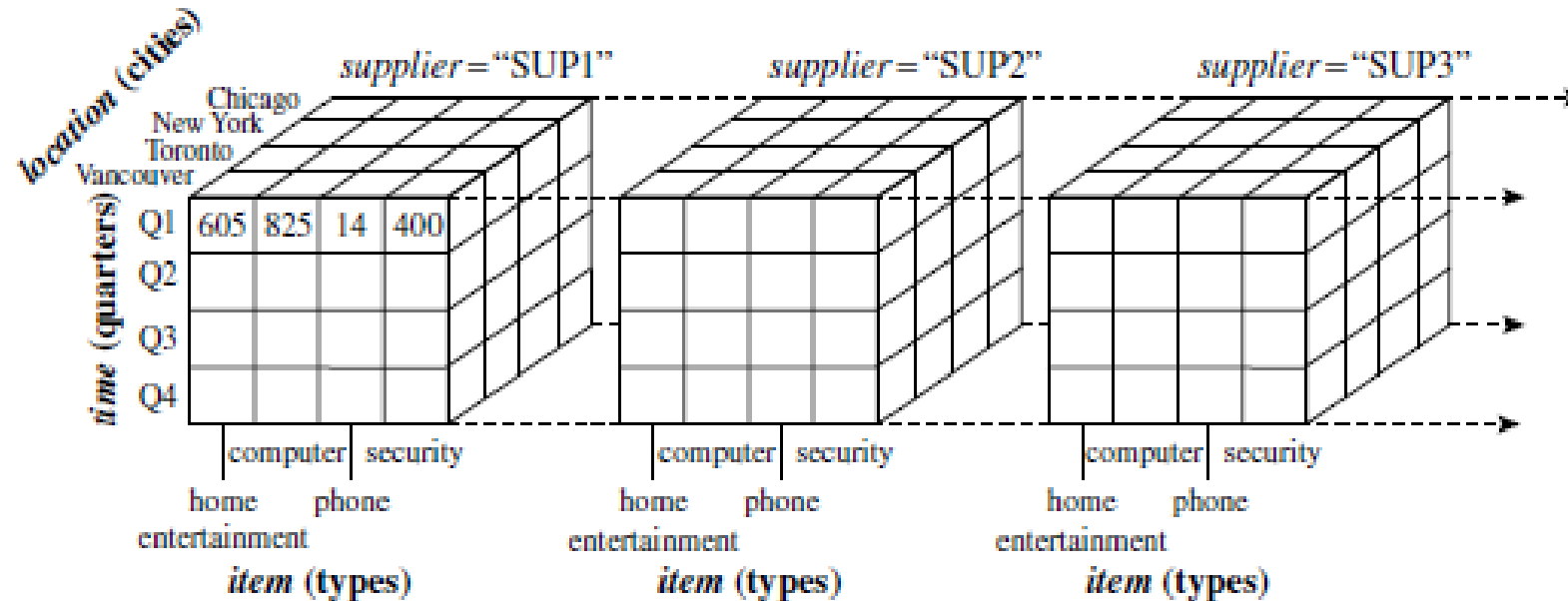
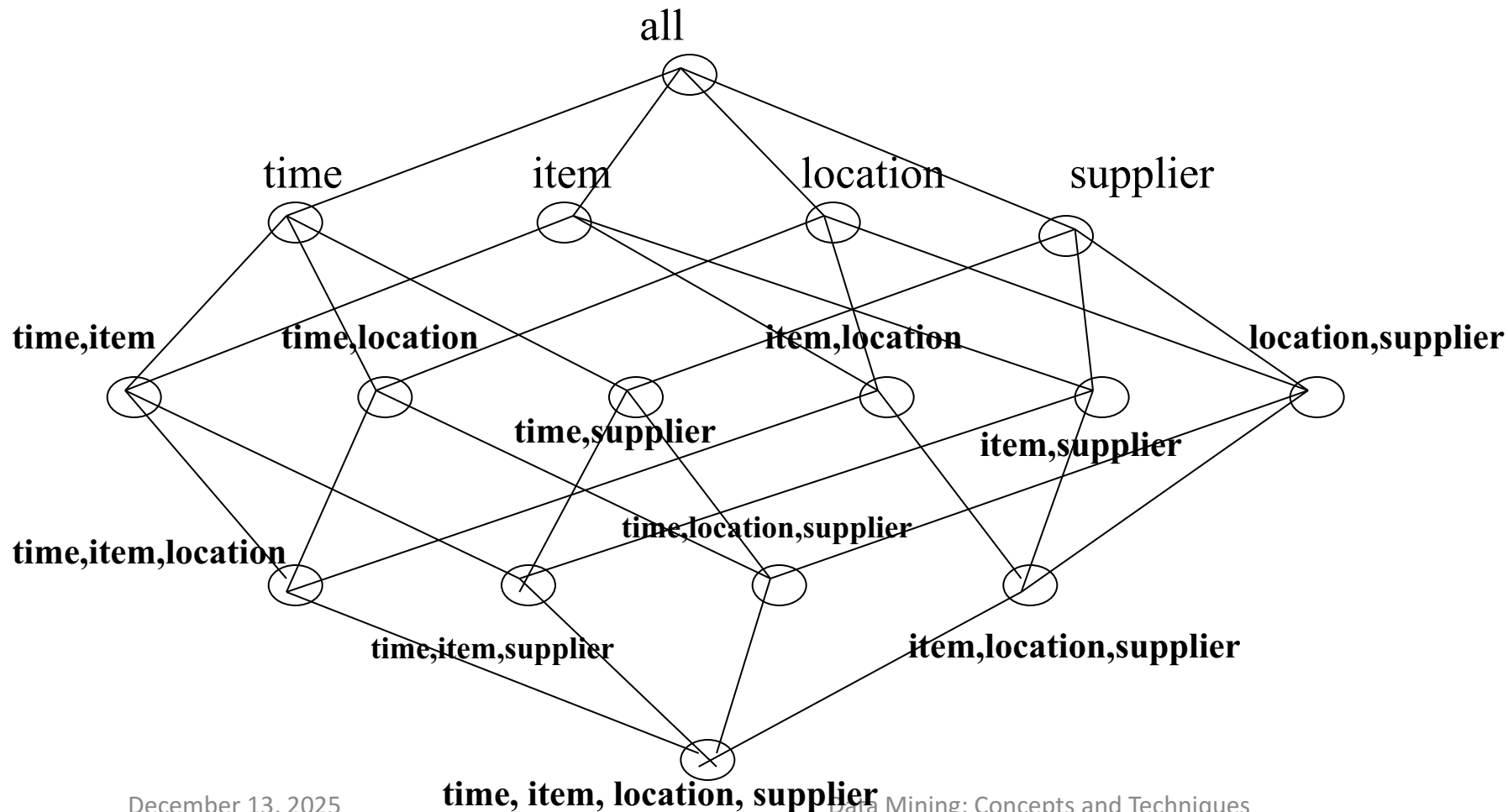


Figure 4.4 A 4-D data cube representation of sales data, according to *time*, *item*, *location*, and *supplier*. The measure displayed is *dollars_sold* (in thousands). For improved readability, only some of the cube values are shown.

Cube: A Lattice of Cuboids



0-D(apex) cuboid

1-D cuboids

2-D cuboids

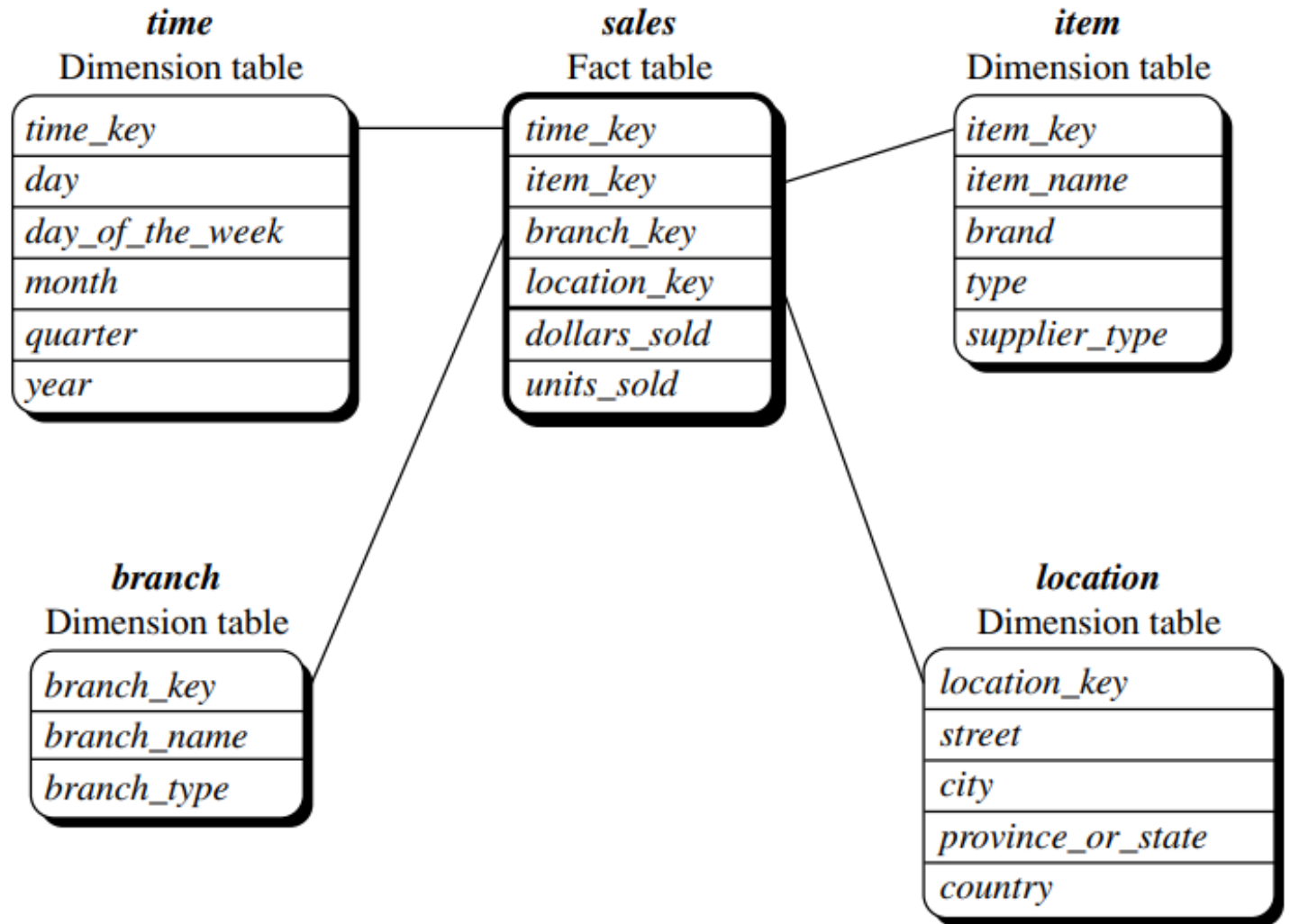
3-D cuboids

4-D(base) cuboid

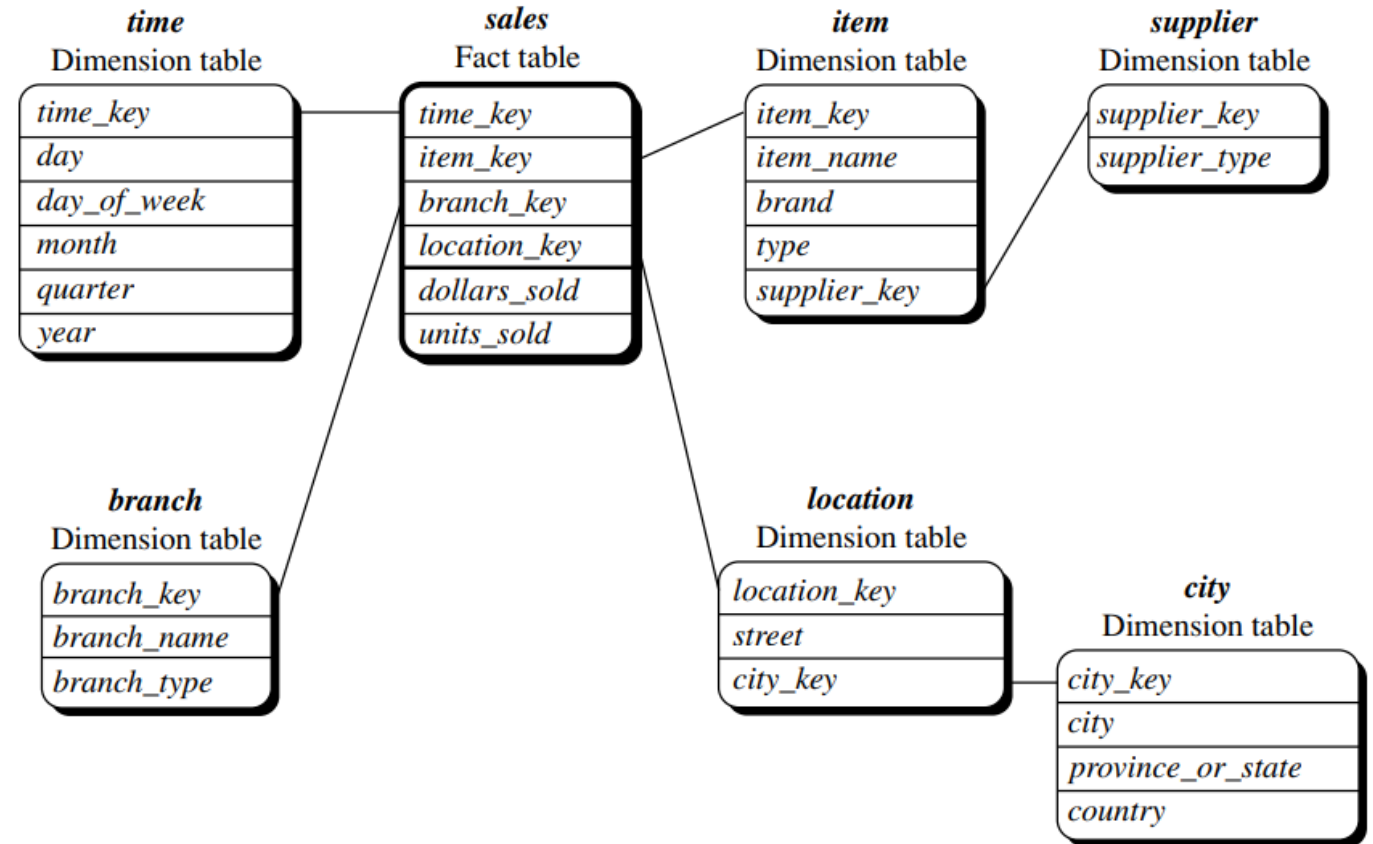
Conceptual Modeling of Data Warehouses

- Modeling data warehouses: dimensions & measures
 - Star schema: A fact table in the middle connected to a set of dimension tables
 - Snowflake schema: A refinement of star schema where some dimensional hierarchy is **normalized** into a set of smaller dimension tables, forming a shape similar to snowflake
 - Fact constellations: Multiple fact tables share dimension tables, viewed as a collection of stars, therefore called **galaxy schema** or fact constellation

Star Schema for Sales Warehouse

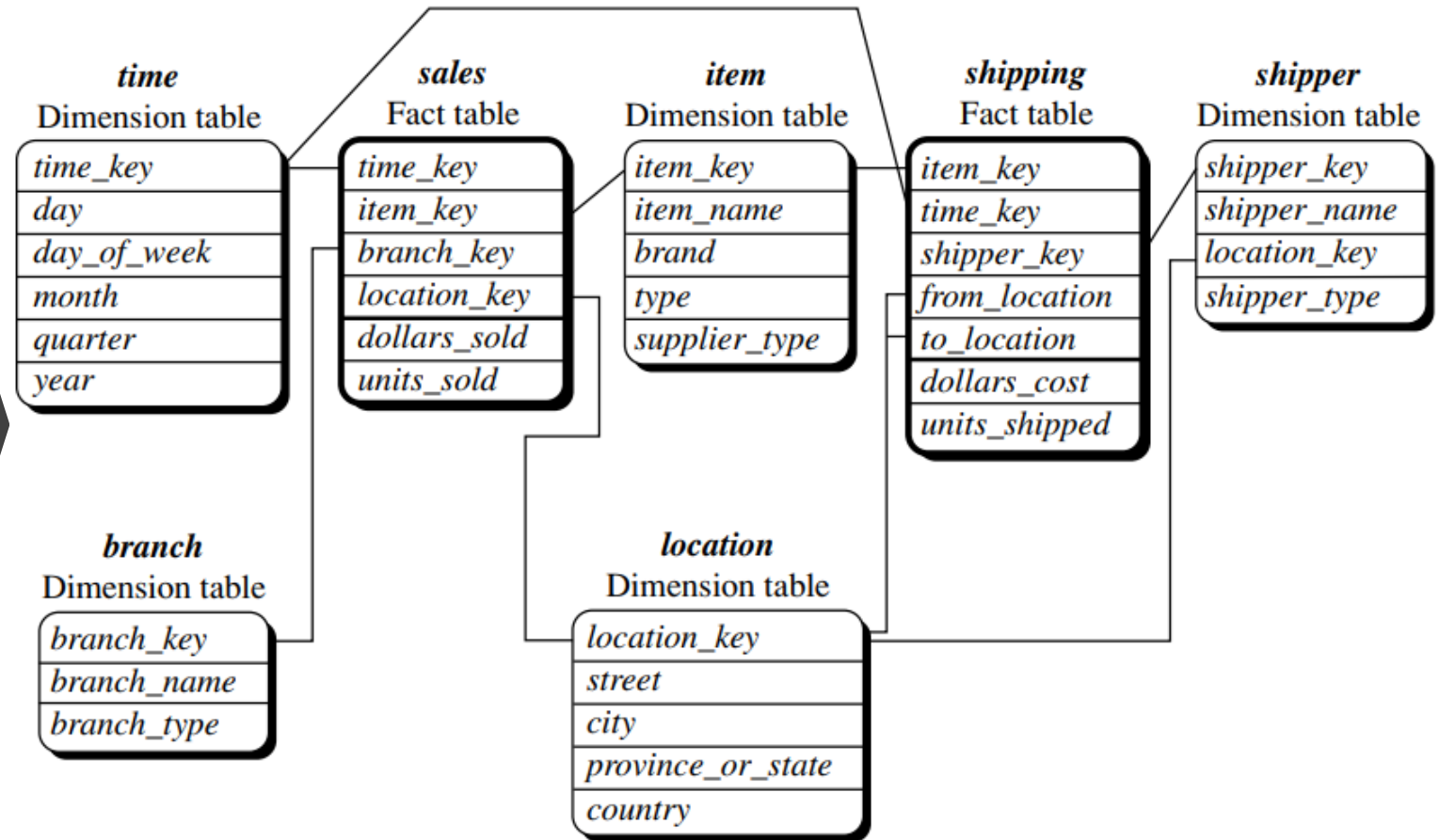


Snowflake Schema for Sales Warehouse



Dimension tables of the snowflake model may be kept in normalized form to reduce redundancies

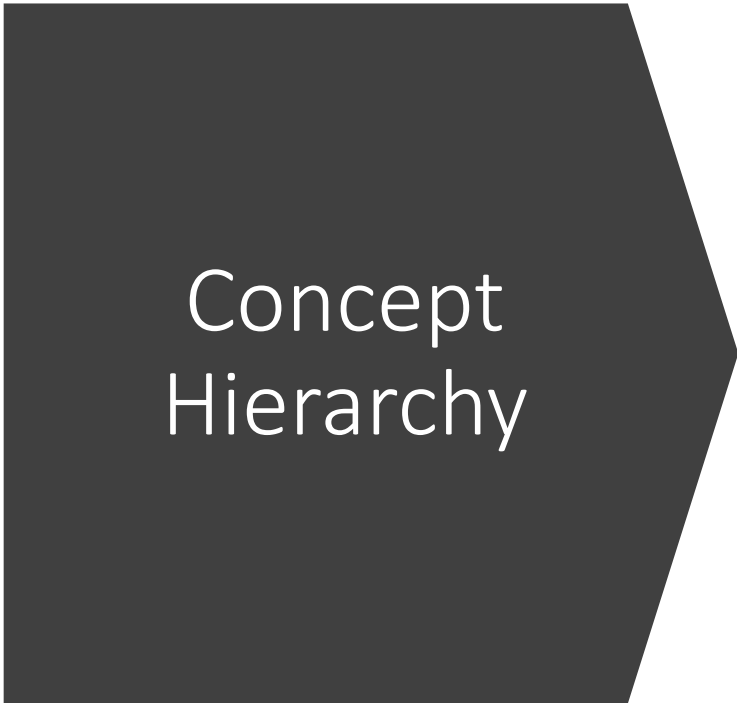
Fast Constellation Schema for Sales and Shipping Warehouse



Concept Hierarchy

- A concept hierarchy defines how specific, low-level items (like cities) are mapped to broader, general categories (like states/provinces and countries) For example:
- **Cities** such as Vancouver, Toronto, New York, and Chicago map to their respective **States/Provinces**
- Those States/Provinces in turn map up to their respective **Countries** (e.g. Canada, USA)

City → State/Province → Country



Concept Hierarchy

Hierarchy Level	Description / Example Values
Lowest Level (Leaf)	City: Vancouver, Toronto, New York, Chicago
Intermediate Level	State / Province: e.g. British Columbia, Ontario, Illinois, New York
Highest Level (Root)	Country: Canada, United States

Hierarchy enables analytics tooling to roll up (aggregate) or drill down between levels —for instance, from cities to countries

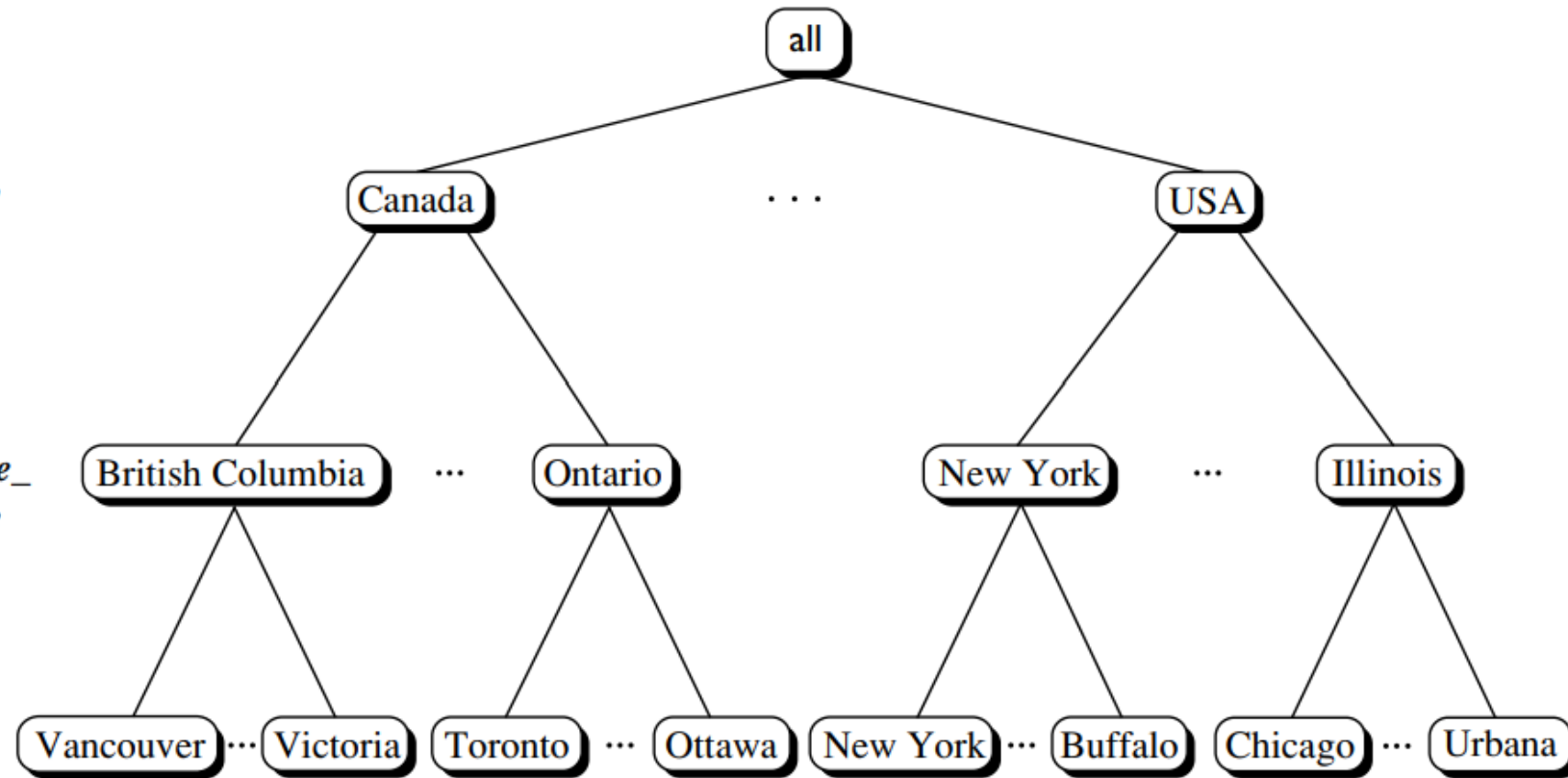
location

all

country

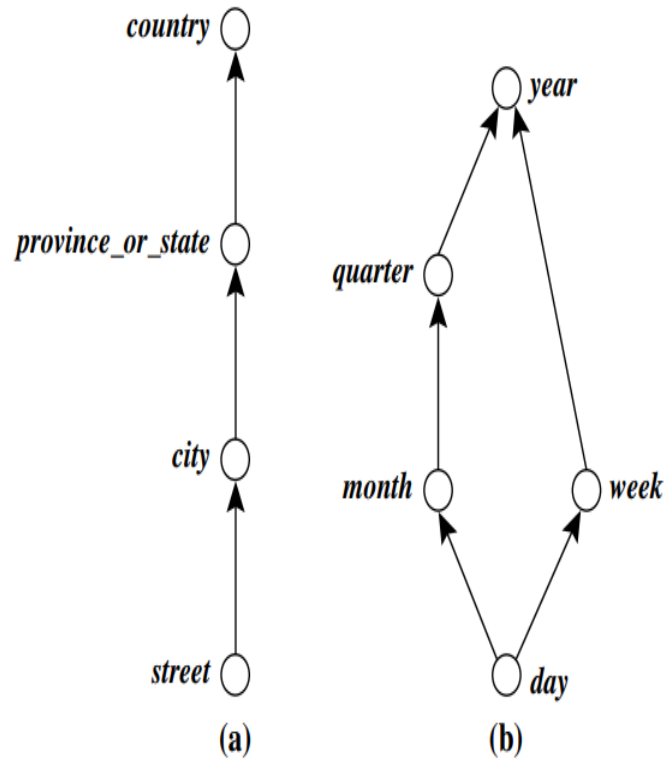
*province_
or_state*

city



4.9 A concept hierarchy for *location*. Due to space limitations, not all of the hierarchy nodes are shown, indicated by ellipses between nodes.

schema hierarchy

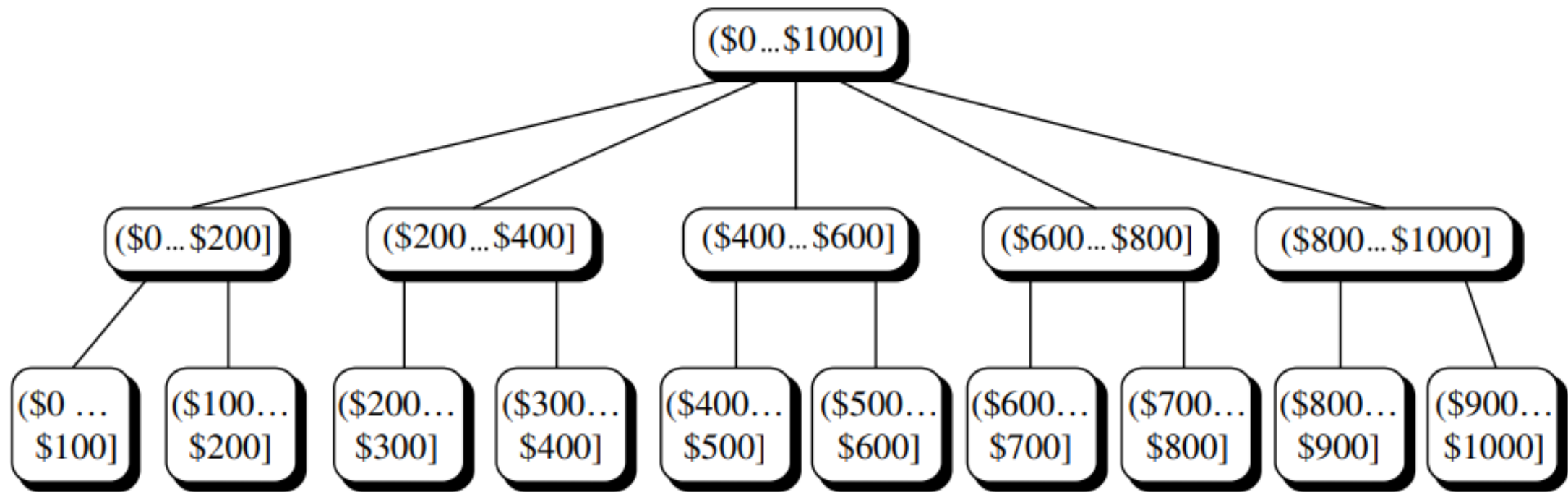


Hierarchical and lattice structures of attributes in warehouse dimensions: (a) a hierarchy for location and (b) a lattice for time.

- Concept hierarchies are implicit within the database schema.
- For example, suppose that the dimension location is described by the attributes number, street, city, province or state, zip code, and country.
- These attributes are related by a total order, forming a concept hierarchy such as “street < city < province or state < country.”

Self Group Hierarchy

- Concept hierarchies may also be defined by discretizing or grouping values for a given dimension or attribute, resulting in a set-grouping hierarchy.
- Total or partial order can be defined among groups of values.
- An example of a set-grouping hierarchy is shown in Figure 4.11 for the dimension price, where an interval $(\$X \dots \$Y]$ denotes the range from $\$X$ (exclusive) to $\$Y$ (inclusive).



A concept hierarchy for *price*.

Enterprise Data Warehouse (EDW)

- An EDW is a centralized repository that collects and manages all historical business data from **various departments and units within an organization.**
- It integrates data from multiple sources, such as ERPs, CRMs, and other operational systems, providing a unified and consistent view of the organization's data.
- This comprehensive approach supports decision-making processes by enabling **cross-functional analysis and reporting.**
- **Key Characteristics:**
 - **Scope:** Organization-wide, encompassing all business areas.
 - **Data:** Detailed and summarized data from multiple sources.
 - **Implementation:** Often complex and time-consuming, requiring extensive business modeling.
 - **Use Cases:** Strategic decision-making, comprehensive reporting, and analytics.

Data Mart

- A Data Mart is a subset of an enterprise data warehouse, **tailored to serve a specific department or business unit, such as sales, finance, or marketing.**
- It focuses on a single subject area, drawing data from fewer sources, which allows for quicker implementation and more targeted analysis.
- **Key Characteristics:**
 - **Scope:** Department-specific, focusing on a particular business area.
 - **Data:** Often summarized and relevant to specific user groups.
 - **Implementation:** Faster and less complex compared to EDWs.

Datamart - Types

- **Dependent Datamart**

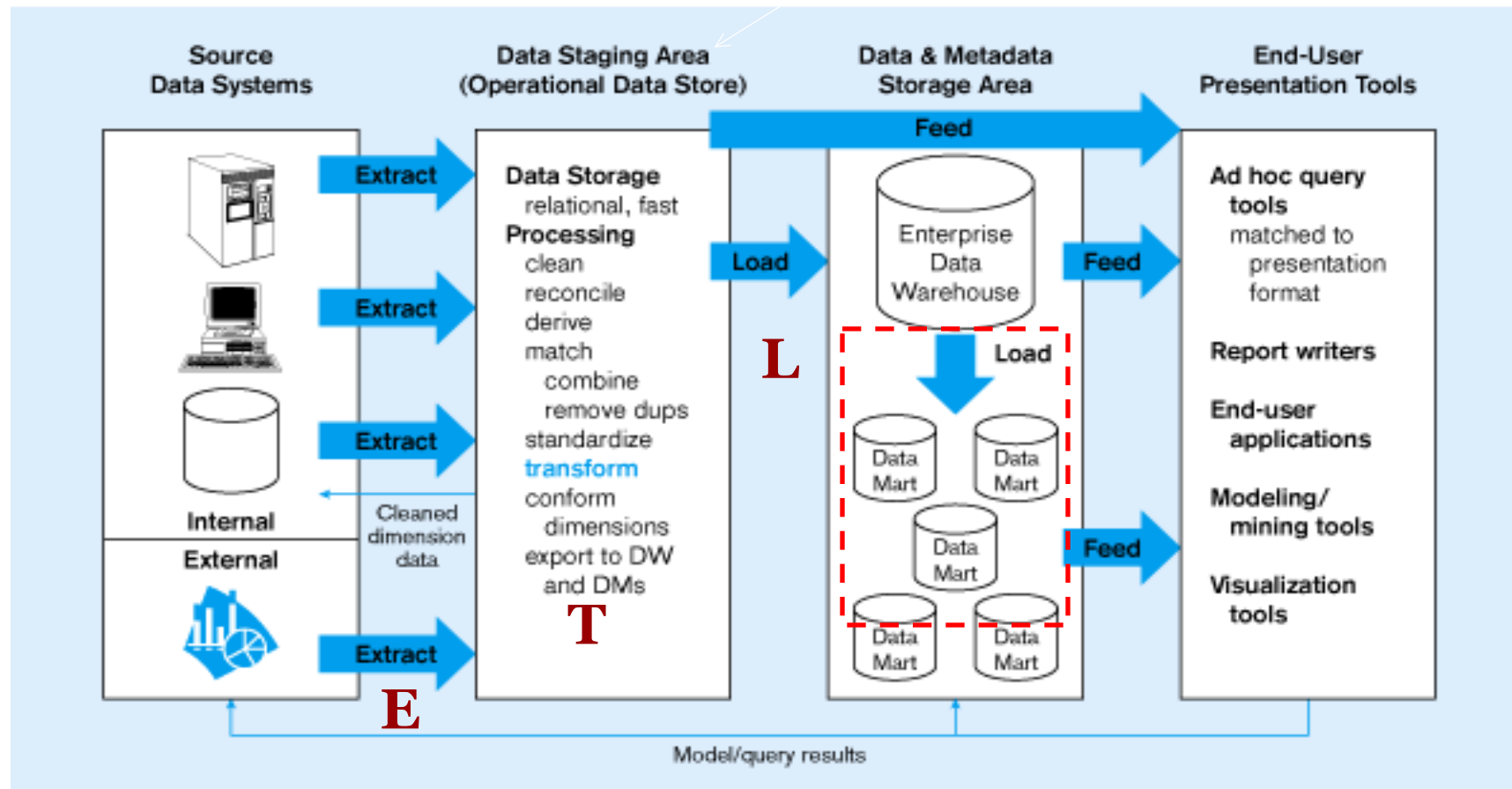
Sourced from a centralized data warehouse

- **Independent Data marts**

Sourced directly from operational systems

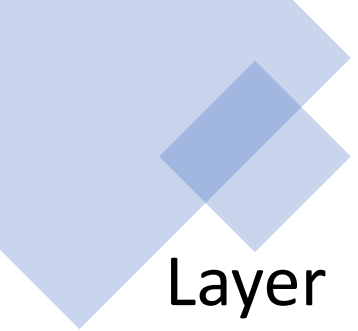
Dependent data mart with operational data store

ODS provides option for obtaining *current* data



Single ETL for
enterprise data warehouse (EDW)

Dependent data marts loaded
from EDW



Layer

Source systems

Staging Area

Enterprise DW & Data Marts

Presentation Tools

Description

Raw data from internal/external sources

Temporary zone for extraction, transformation, cleaning

Final structured storage for broad or departmental needs

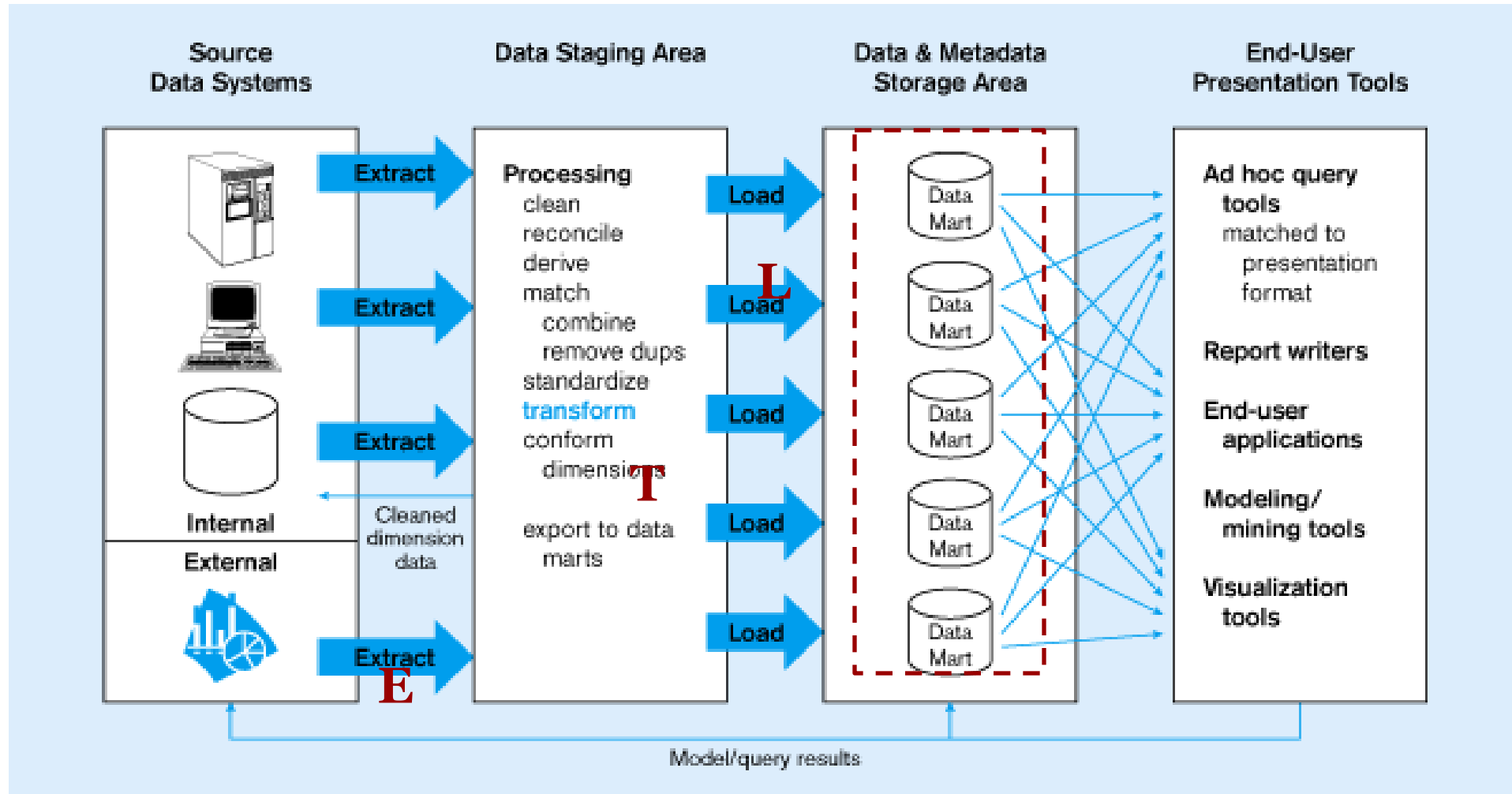
BI, reporting, analytics tools for end-users



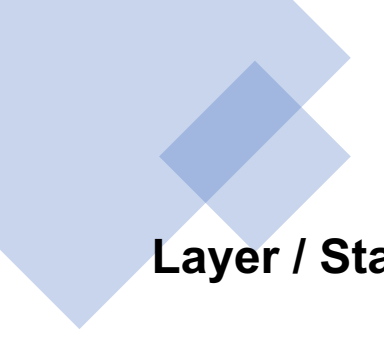
Independent data mart

Data marts:

Mini-warehouses, limited in scope



Separate ETL for each *independent* data mart



Layer / Stage

Source Data Systems

Function & Purpose

Raw data originates here—from internal systems (e.g. ERP, CRM, OLTP DBs) and external sources (APIs, flat files, partner feeds).

Activities Performed

Extract raw data for staging

Data Staging Area

A temporary "landing zone" where raw data is consolidated and made ready for warehousing.

Clean, reconcile, remove duplicates, standardize, derive fields, conform dimensions, match, combine. Acts as buffer and audit zone.

Data & Metadata Storage Area

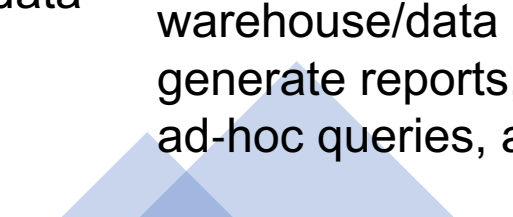
This is the structured storage layer: **enterprise warehouse** or **data marts**, enriched with metadata (schema info, ETL logic).

Load cleaned & transformed data into data warehouse/data marts; store metadata for governance and lineage.

End-User Presentation Tools

Tools that provide access to data for reporting, analysis, visualization, and mining.

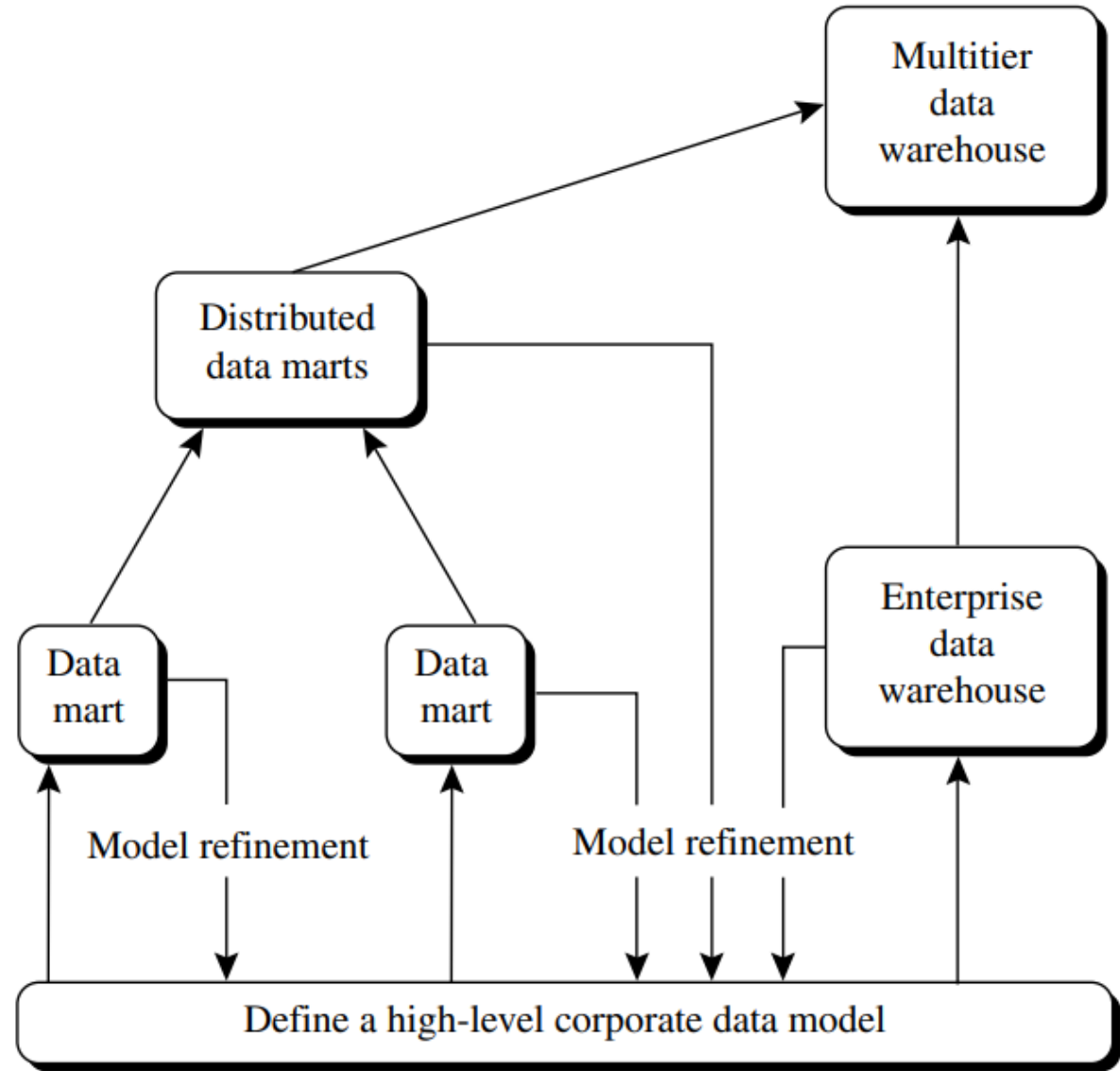
Connects to data warehouse/data marts to generate reports, dashboards, ad-hoc queries, and analytics.



Virtual Warehouse

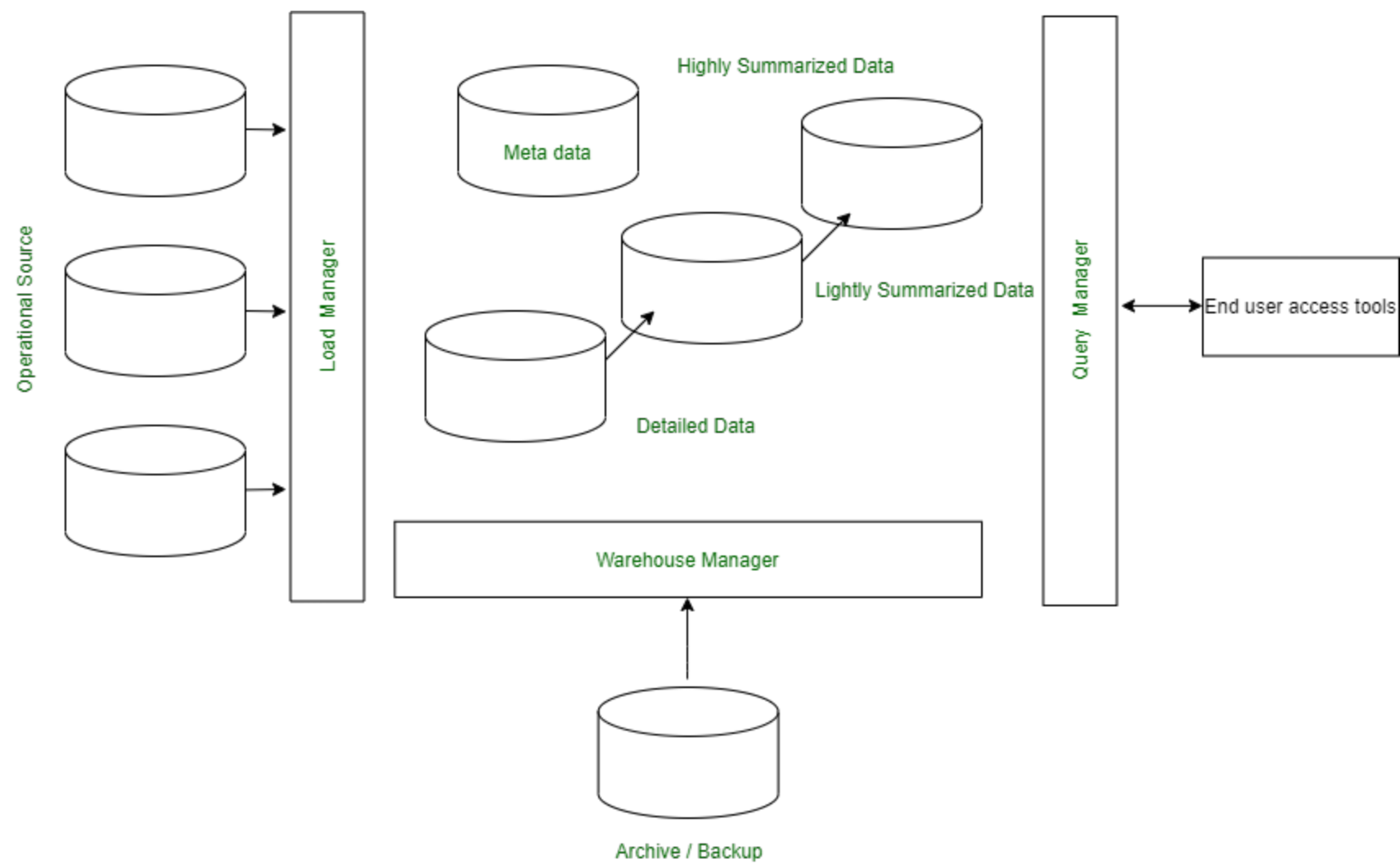
- A Virtual Warehouse provides a logical view of data from **multiple sources without physically storing the data in a centralized repository**.
- It integrates data on **an as-needed basis**, allowing users to access and query data from disparate systems in real-time.
- **Key Characteristics:**
 - **Scope:** Logical integration across various data sources.
 - **Data:** Accessed in real-time without physical consolidation.
 - **Implementation:** Simpler setup but may require robust infrastructure for performance.
 - **Use Cases:** Situations where data centralization is impractical, real-time data access, and scenarios requiring minimal data redundancy

A recommended
approach for
data warehouse
development



Process Architecture

- Data warehouse architecture integrates multiple processes.
- Architecture varies between organizations, but core functionality remains.
- Extract & Load → Load Manager
- Clean & Transform → Warehouse Manager
- Backup & Archive → Warehouse Manager
- Query Management → Query Manager



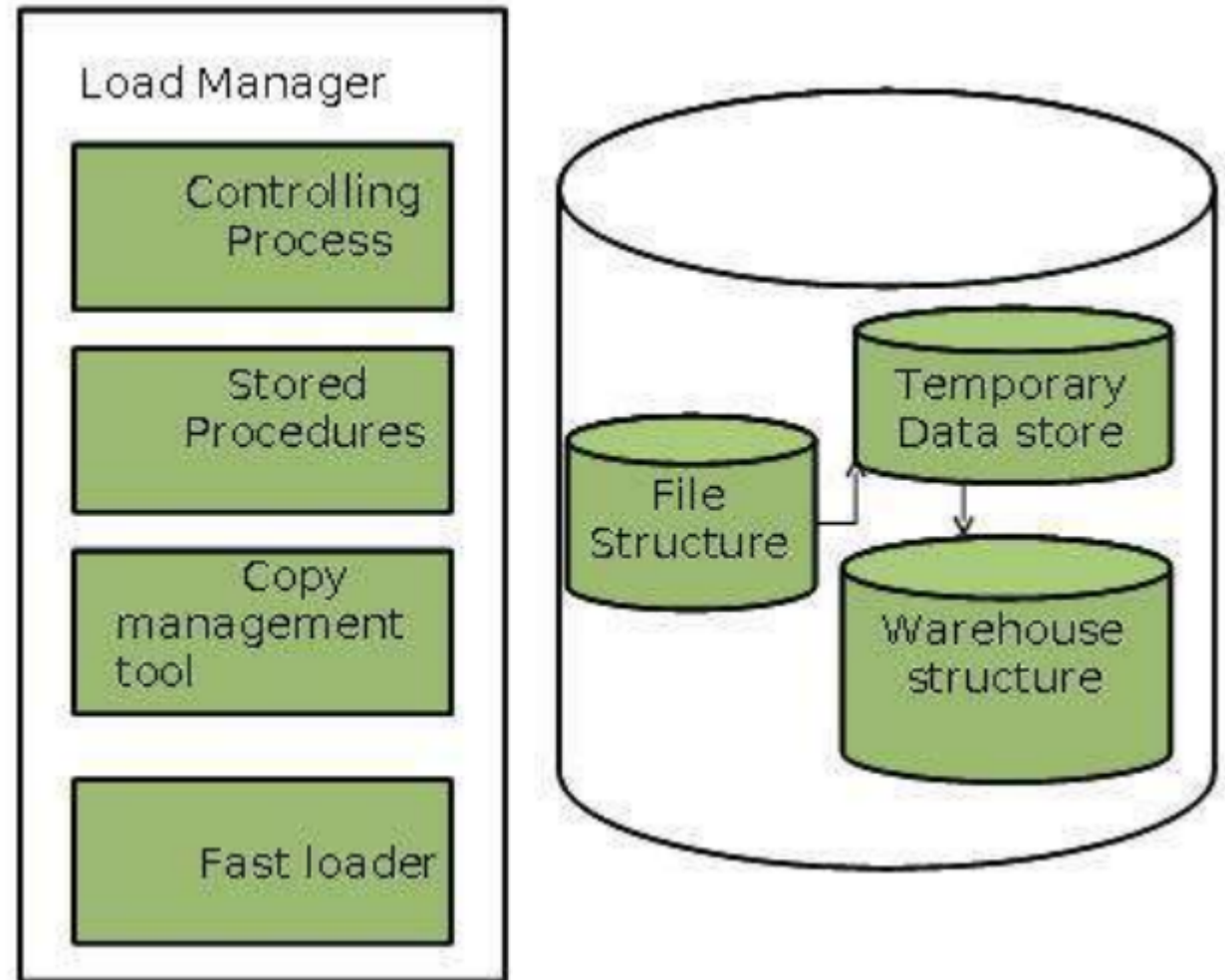
Load Manager

- Responsible for extraction and loading of data
- Complexity varies by system
- Performs simple transformations

Load Manager Architecture

Components:

- Extraction from source systems
- Fast loading to staging area
- Simple transformation
- Uses Gateways (e.g., ODBC, JDBC)
- Temporary data store



Extract Data from Source

- Sources: Operational DBs, external data providers
- Gateways enable SQL generation at server
- Examples: ODBC, JDBC

Fast Load

- Importance of load performance
- Transformations delay loading
- Efficient to load raw data first
- Gateways inefficient for large volumes

Simple Transformations

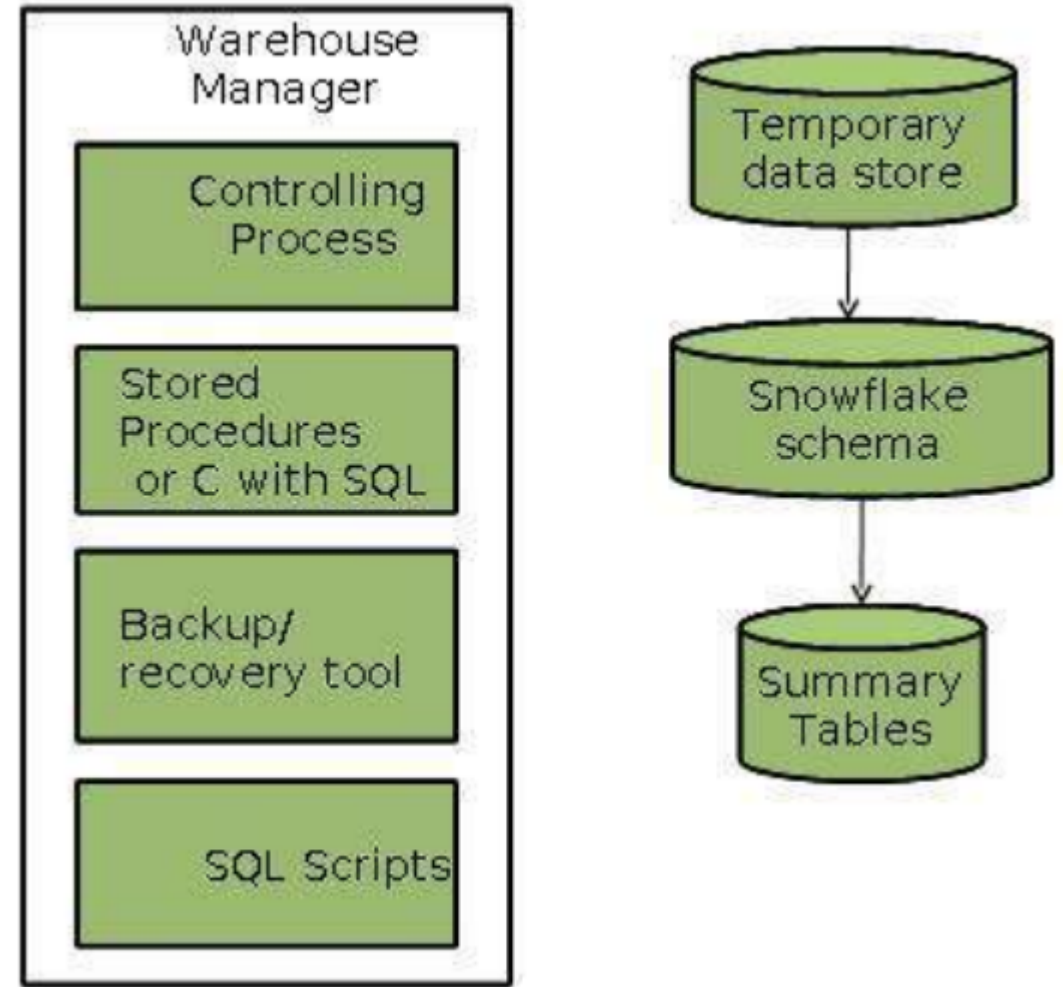
- Remove unnecessary columns
- Convert to required data types
- Example: EPOS sales transaction processing

Warehouse Manager – Overview

- Manages the warehouse operations
- Uses system software, shell scripts, SQL, C programs
- Varies in size and complexity

Warehouse Manager Architecture

- **Key operations:**
- Consistency checks
- Transform & merge data
- Create indexes/views
- Generate & update aggregations
- Backup/archive



Functions of Warehouse Manager

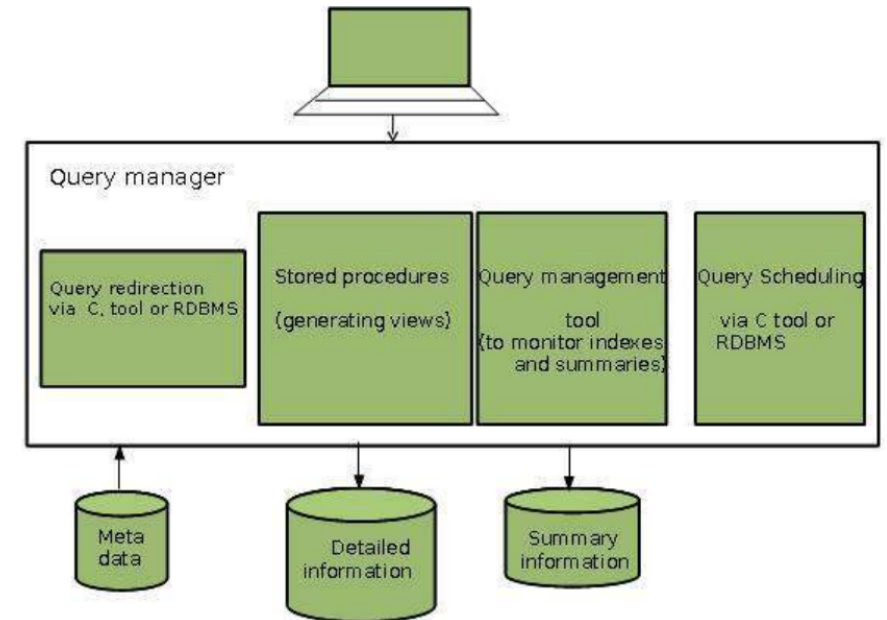
- Performs data consistency and integrity checks
- Creates indexes, views, aggregations
- Transforms and merges data
- Backs up and archives data
- Analyzes query profiles

Query Manager – Overview

- Directs queries to appropriate tables
- Schedules query execution
- Speeds up query response time

Query Manager Architecture

- Components:
- Query redirection (C tool/RDBMS)
- Stored procedures
- Query scheduling tools (internal or 3rd party)



Functions of Query Manager

- Presents user-friendly data
- Schedules user queries
- Stores query profiles for optimization
- Aids warehouse manager in tuning (indexes & aggregations)

Summary-Process Architecture

- Load Manager: Extract → Load → Simple Transform
- Warehouse Manager: Manage → Transform → Archive
- Query Manager: Direct → Schedule → Optimize
- Each component ensures efficient data warehouse operations